NAAN MUDHALVAN • PROJECT PRACTICAL & VIVA

C!

ComicCraft

AI Comic Story Creator Using Gemini Models

A resilient FastAPI web application that converts a short story brief into a structured, visually consistent multi-panel comic book with responsive browser preview, REST JSON API, and downloadable A4 PDF export.

Python 3.11+ | FastAPI + Uvicorn | Google Gemini JSON

Pydantic v2 | SDXL / Diffusers | fpdf2 Unicode PDF

END-TO-END AI PIPELINE

01 **Story Brief Input**
Idea, Hero, Tone, 3–8 Panels

02 **2-Stage Gemini JSON**
Outline + Character Bible + Script

03 **Pluggable Panel Art**
Placeholder / HF SDXL / Diffusers

04 **Web & A4 PDF Export**
UUID Workspace + fpdf2 Output

Department of Computer Science | Project Demonstration & Viva

Zero-Credential Offline Demo Ready • 11/11 Pytest Verified

Abstract & Project Overview

Turning a compact creative brief into a structured, illustrated, downloadable comic book

EXECUTIVE ABSTRACT

- **Compact Creative Brief:** Users enter a story idea, main character, setting, tone, art style, and panel count (3–8 panels) via Web UI or REST JSON API.
- **Two-Stage Gemini AI:** Stage 1 creates a JSON outline and Character Bible; Stage 2 expands it into captions, narration, dialogue, and image prompts.
- **Schema-Validated Data:** All inputs and AI JSON responses are strictly validated using Pydantic v2 models instead of fragile text splitting.
- **Pluggable Artwork & PDF:** Generates per-panel PNGs (Placeholder, Hugging Face SDXL, or Local Diffusers) and compiles a Unicode A4 PDF via fpdf2.
- **100% Offline Resilience:** Includes exponential-backoff retries, fallback models, and an offline deterministic Demo Mode requiring zero API keys.

FOUR-STAGE GENERATION WORKFLOW

01 **PLAN • Story Outline & Character Bible**

Gemini Outline model plans exact N sequential panels and locks hero visual traits in JSON.

02 **WRITE • Full Comic Script**

Gemini Story model writes panel captions, narration, 0–3 dialogue lines, and art prompts.

03 **ILLUSTRATE • Seeded Panel Artwork**

SHA-256 deterministic seed produces 1 PNG per panel via Placeholder, HF SDXL, or Diffusers.

04 **PUBLISH • Web Preview & A4 PDF**

Builds Cover + Panel PDF (fpdf2) and saves atomic comic.json inside a UUID workspace.

Problem Statement & Core Objectives

Why manual or naive AI comic scripts fail — and the engineering goals ComicCraft achieves

PROBLEMS IN EXISTING / NAIVE APPROACHES

1. Multi-Skill Bottleneck

Creating a comic manually requires separate writing, illustration, and page-layout skills, causing beginners to abandon projects.

2. Brittle Plain-Text Parsing

Naive AI scripts split free-form chat text on markers like '**Panel' — minor formatting drift crashes the parser.

3. Character Visual Drift

Generating panel images independently causes the main character's face, clothes, and colors to change every panel.

4. Hard Failures & Paid Key Lock-In

API rate-limits (429), retired model names (404), or missing paid credentials break the entire application.

COMICCRAFT PROJECT OBJECTIVES

1. Unified Single-Brief Studio

Generate complete 3–8 panel comics from one browser form or versioned REST JSON API (/api/v1/comics).

2. Strict JSON Schema Validation

Use Gemini JSON mode parsed into Pydantic v2 contracts that enforce exact panel counts and numbering.

3. Character Bible Continuity

Lock hero appearance in Stage 1 and repeat visual traits + negative prompts across every panel image.

4. Graceful Offline & Fallback Support

Provide retries, fallback model chains, offline Demo Writer, and Pillow Placeholder art for 100% uptime.

Existing System vs Proposed System

Direct architectural comparison between conventional/fragile scripts and ComicCraft

Dimension	Existing / Naive AI System	Proposed System (ComicCraft)
Workflow Pipeline	Disconnected tools for script writing, illustration, and manual PDF layout.	**Single integrated FastAPI pipeline from user brief to preview and A4 PDF.**
AI Output Parsing	Free-form prose split by string markers; breaks on slight format changes.	**Gemini JSON mode validated strictly by Pydantic v2 (ComicOutline & ComicScript).**
Visual Continuity	Ad-hoc per-panel prompts cause hero's face and outfit to drift.	**Stage-1 Character Bible repeated in every panel prompt + negative prompts.**
Fault Tolerance	Single 429 rate-limit, 404 model rename, or missing key crashes the app.	**Exponential backoff + fallback model chain + offline Demo & Placeholder modes.**
Storage & Security	Unsafe prompt filenames, overwrites, and no reopenable state.	**32-hex UUID workspace, regex path guard, and atomic comic.json manifest.**

Key Engineering Upgrade: Every AI response is treated as schema-validated structured JSON data — never fragile prose — with honest provider badges and warnings.

Technology Stack & System Requirements

Verified open-source libraries, AI SDKs, and lightweight runtime specifications

BACKEND & SERVER

Python 3.11+ · FastAPI · Uvicorn

High-speed ASGI web framework with automatic Swagger /docs and run_in_threadpool() for blocking tasks.

VALIDATION & CONFIG

Pydantic v2 · pydantic-settings

Validates form/JSON inputs, AI JSON contracts, and loads .env API keys securely into SecretStr fields.

AI STORY ENGINE

Google Gemini · google-genai SDK

Two-stage JSON generation (Outline -> Story models) plus deterministic offline DemoStoryProvider.

IMAGE PROVIDERS

Pillow · HuggingFace · Diffusers

Pluggable protocol supporting local Placeholder PNGs, Cloud SDXL inference, and lazy Local Stable Diffusion.

PDF & FRONTEND UI

fpdf2 + DejaVu · Jinja2 + JS

Unicode-safe A4 PDF builder (cover + panel pages) and responsive framework-free HTML5/CSS3/JS UI.

TESTING & DEVOPS

pytest (11 Tests) · Ruff · Docker

~80% offline test coverage (zero paid API calls), clean static linting, and Docker Compose packaging.

MINIMUM SYSTEM REQUIREMENTS (RUNS FULLY OFFLINE WITHOUT GPU)

OS: Windows / Linux / macOS / Android Termux • CPU: Core i3 / Ryzen 3+ • RAM: 4 GB (8 GB rec.) • Disk: ~250 MB • Database: None (Local UUID Filesystem)

System Architecture

Layered service-oriented architecture from client request to persisted UUID comic workspace

Web Browser Client

Jinja2 Form (GET /, POST /generate) • Preview & Download

REST JSON API Client

POST /api/v1/comics • GET /api/v1/comics/{id}

DevOps & Diagnostics

GET /health Probe • Interactive OpenAPI /docs

▼

FASTAPI APPLICATION LAYER (app/main.py & app/routes.py)

Pydantic ComicPrompt Validation • Security Headers (nosniff, SAMEORIGIN) • Async-Safe run_in_threadpool() • Error Handlers

▼

COMIC SERVICE ORCHESTRATOR (app/services/comic_service.py)

Creates Workspace -> Generates Story -> Renders Panel PNGs -> Builds PDF -> Saves Manifest

▼

ResilientStoryService

services/llm.py

- 2-Stage Gemini JSON
- Retry + Backoff + Jitter
- Model Fallback Chain
- Offline Demo Writer

ResilientImageService

services/images.py

- SHA-256 Panel Seed
- Pillow Placeholder Art
- Hugging Face SDXL API
- Local Diffusers SD v1.5

PDFService

services/pdf_service.py

- fpdf2 A4 Layout
- Dark Styled Cover Page
- 180×145mm Aspect Fit
- Bundled DejaVu Fonts

ComicRepository

services/repository.py

- storage/comics/<uuid>/
- Regex ^[a-f0-9]{32}$
- panels/*.png & comic.pdf
- Atomic comic.json Save

Two-Stage Gemini Workflow & Character Continuity

How structured JSON generation and a Character Bible keep multi-panel stories coherent

STEP 1

Validated Brief

ComicPrompt

- Story idea (10–1500 chars)
- Hero name & setting
- Tone & art style
- Panel count: 3 to 8
- Blocks control chars

→

STEP 2

Stage 1: Outline

gemini-3.5-flash-lite

- JSON MIME mode forced
- Generates Comic Title
- Creates Character Bible
- Plans N sequential scenes
- Validated by ComicOutline

→

STEP 3

Stage 2: Script

gemini-3.5-flash

- Receives approved outline
- Panel caption & narration
- 0–3 dialogue lines/panel
- Repeats Character Bible
- Validated by ComicScript

→

STEP 4

Seeded Panel Art

ImageProvider Protocol

- SHA-256 deterministic seed
- Negative prompts applied
- No text bubbles inside art
- Saves panel-1..N.png
- Feeds PDF & Web preview

WHY TWO GEMINI STAGES?

Single-prompt generation mixes story planning with dialogue writing, causing missing panels or broken pacing. Splitting Outline (fast lite model) and Script (story model) guarantees exact panel counts and richer storytelling.

HOW CHARACTER CONTINUITY WORKS

Stage 1 produces a 'character_bible' (fixed face, hair, outfit, and colors). Stage 2 embeds those exact traits into every panel's image_prompt, while negative prompts ban 'inconsistent clothing' and stray text bubbles.

Resilience, Graceful Fallbacks & Security

Engineered so transient API failures, missing keys, or bad inputs never crash the application

1. SMART GEMINI RETRY & FALLBACK

- **Transient 429 / 5xx Errors:** Retries up to GEMINI_RETRY_ATTEMPTS using bounded exponential backoff (base × 2^attempt) plus random jitter.
- **400 / 404 Invalid Model IDs:** Skips waiting and switches immediately to configured fallback Gemini models in .env.
- **Offline Story Fallback:** If all live models fail and ALLOW_AI_FALLBACK=true, DemoStoryProvider completes the comic with a visible warning.

2. PER-PANEL IMAGE PROTECTION

- **Pluggable Provider Layer:** Supports Placeholder (Pillow), Hugging Face Hosted SDXL, and lazy-loaded Local Stable Diffusion.
- **Panel-Level Isolation:** If panel #3 fails on Hugging Face, the whole comic is NOT lost—only panel #3 falls back to a labeled Placeholder PNG.
- **Honest Provider Metadata:** ProviderInfo records exact models used, image providers, used_fallback=true, and user-visible warnings.

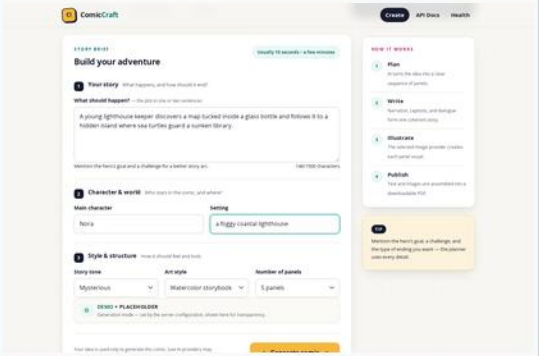
3. STORAGE & SECURITY GUARDS

- **Path Traversal Protection:** Every comic_id must match ^[a-f0-9]{32}$ before any disk read/write, blocking '../' traversal attacks.
- **Atomic Manifest Writes:** Writes comic.json.tmp first and atomically replaces comic.json so crashed writes never corrupt saved comics.
- **Secret & Header Hygiene:** API keys stored as Pydantic SecretStr in .env; auto-deletes failed workspaces; sets nosniff & SAMEORIGIN headers.

Application Output & Live Screenshots

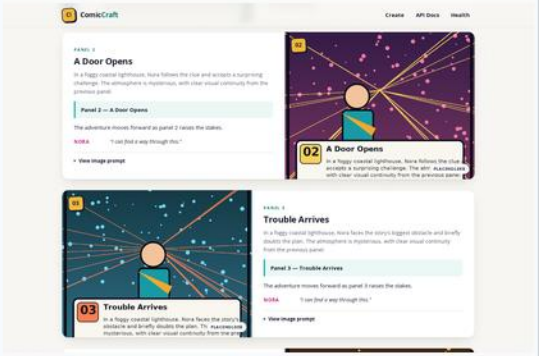
Verified captures from the running ComicCraft application — Web Studio, Panels & A4 PDF

1. Interactive Story Brief Form



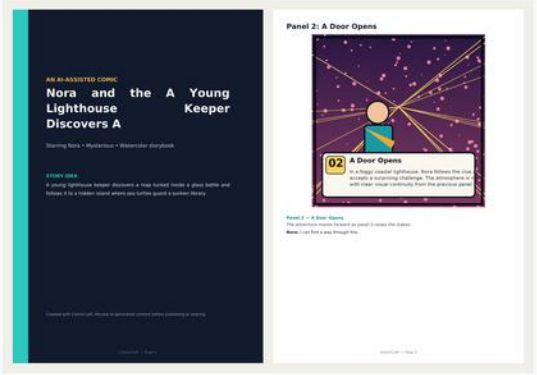
Grouped sections (Story, Character & World, Style & 3–8 Panels) with live character counter and active mode card.

2. Generated Comic Panels UI



Alternating responsive cards pairing panel artwork, scene summary, caption, narration, dialogue, and provider badge.

3. Exported A4 Comic PDF



Built with fpdf2 & DejaVu Unicode fonts: styled dark cover page + one aspect-preserved page per panel.

Testing, Future Enhancements & Conclusion

Verified offline test suite, roadmap for production scale, and final summary

AUTOMATED TESTING & QA

- **11 / 11 Pytest Suite PASS:** Runs 100% offline using demo + placeholder providers and mocked Gemini clients—zero paid API calls.
- **~80% Code Coverage:** 99% coverage on pdf_service.py, 95% on config.py, and 93% on main.py.
- **End-to-End Verified:** Tested /health, HTML form flow, REST 201 JSON lifecycle, PDF export, 422/404 handling, and 503/404 fallbacks.

FUTURE ENHANCEMENTS

- **Async Background Queue:** Add Celery/Redis task queues with live progress polling for multi-minute diffusion generations.
- **Cloud Auth & Storage:** Add user login, quotas, PostgreSQL metadata, and AWS S3 object storage for multi-user deployment.
- **Rich Editing & Regional UI:** Support single-panel regeneration, multi-panel comic grid layouts, and Tamil language comic scripts.

CONCLUSION

- **Complete Integrated Product:** Transforms a one-line story idea into a previewed, illustrated, A4 PDF comic book in one workflow.
- **Production-Grade AI Patterns:** Replaces brittle text splitting with Pydantic v2 JSON validation and Character Bible visual continuity.
- **Always Demonstrable:** Layered retries and offline fallbacks guarantee smooth operation with or without external API credentials.

Thank You! — Questions & Live Demo Welcome

Run Live Demo: uvicorn app.main:app --host 0.0.0.0 --port 8000

Routes: / · /comics/{id} · /download/{id} · /docs